Branching and Looping in C

Introduction

In C programming, the flow of execution of instructions is not always sequential. We need ways to:

1. **Branch (Decision making):** Choose one path from multiple options.
2. **Loop (Iteration):** Repeat a block of code until a condition is satisfied.

Branching (Decision Making Statements)

1. Simple if Statement

Executes a block of code only if a given condition is true.

Syntax:

```
if (condition) {
   // code to execute if condition is true
}
```

Example:

```
int age = 20;
if (age >= 18) {
   printf("You are eligible to vote.");
}
```

---

2. if-else Statement

Provides two options – one when condition is true, another when false.

Syntax:

```
if (condition) {
   // executes if condition is true
} else {
   // executes if condition is false
}
```

Example:

```
int num = 7;
if (num % 2 == 0)
```

```
   printf("Even number");
else
   printf("Odd number");
```

3. Nested if Statement

One if inside another if.

Example:

```
int marks = 85;
if (marks >= 50) {
   if (marks >= 80)
      printf("Distinction");
   else
      printf("Pass");
} else {
   printf("Fail");
}
```

4. else-if Ladder

Used to check multiple conditions sequentially.

Syntax:

```
if (condition1) {
   // block 1
} else if (condition2) {
   // block 2
} else if (condition3) {
   // block 3
} else {
   // default block
}
```

Example:

```
int marks = 65;
if (marks >= 80)
   printf("Grade A");
else if (marks >= 60)
   printf("Grade B");
else if (marks >= 40)
   printf("Grade C");
else
   printf("Fail");
```

5. switch Statement

Used when a variable is compared against multiple constant values.

Syntax:

```
switch (expression) {
   case value1:
      // code
      break;
   case value2:
      // code
      break;
   default:
      // code
}
```

Example:

```
int day = 3;
switch(day) {
   case 1: printf("Monday"); break;
   case 2: printf("Tuesday"); break;
   case 3: printf("Wednesday"); break;
   default: printf("Invalid day");
}
```

6. Ternary (? :) Operator

Shorthand for if-else.

Syntax:

```
(condition) ? expression1 : expression2;
```

Example:

```
int age = 16;
(age >= 18) ? printf("Adult") : printf("Minor");
```

7. goto Statement

Transfers control to a labeled statement (not recommended in modern practice).

Example:

```
int i = 1;
start:
printf("%d ", i);
i++;
if (i <= 5) goto start;
```

Looping (Iteration Statements)

1. while Loop

Condition is checked first, then loop body executes.

Syntax:

```
while (condition) {
   // code
}
```

Example:

```
int i = 1;
while (i <= 5) {
   printf("%d ", i);
   i++;
}
```

2. do-while Loop

Executes at least once because condition is checked **after** execution.

Syntax:

```
do {
   // code
} while (condition);
```

Example:

```
int i = 1;
do {
   printf("%d ", i);
   i++;
} while (i <= 5);
```

3. for Loop

Compact form of loop.

Syntax:

```
for (initialization; condition; update) {
   // code
}
```

Example:

```
for (int i = 1; i <= 5; i++) {
    printf("%d ", i);
}
```

Decrementing Example:

```
for (int i = 5; i >= 1; i--) {
    printf("%d ", i);
}
```

---

4. Nested Loops

Loop inside another loop.

Example:

```
for (int r = 1; r <= 3; r++) {
    for (int c = 1; c <= 3; c++) {
        printf("%d ", r * c);
    }
    printf("\\n");
}
```

Jump Statements in Loops

1. break Statement

Terminates the loop immediately.

Example:

```
for (int i = 1; i <= 10; i++) {
    if (i == 5) break;
    printf("%d ", i);
}
```

---

2. continue Statement

Skips the current iteration and continues with the next.

Example:

```
for (int i = 1; i <= 5; i++) {
    if (i == 3) continue;
    printf("%d ", i);
}
```